**VINH UNIVERSITY**
**SCHOOL OF ENGINEERING AND TECHNOLOGY**

# COURSE PROJECT REPORT

**Subject: E-Commerce**

# BUILDING A MULTI-TENANT SAAS PLATFORM FOR FOOTBALL FIELD MANAGEMENT INTEGRATING AI AND VNPAY

**Performed by:** **TRAN DUC LUONG**
**Student ID:** **225748020110186**
**Major:** Information Technology
**Specialization:** Software Engineering

**Supervisor:** **M.Sc. Le Van Thanh**

Nghe An, 05/2026

# COURSE PROJECT TOPIC

Full name: Tran Duc Luong Student ID: 225748020110186
Cohort: 63 School: School of Engineering and Technology

1. Topic

**Building a Multi-Tenant SaaS Platform for Football Field Management integrating AI and VNPay**

2. Content

Researching and developing a SaaS system with multi-tenant architecture, integrating Google Gemini AI and VNPay payment gateway for football field management.

3. Topic assignment date: March 2026

4. Topic complete date: May 2026

May 2026

**INSTRUCTOR**
**M.Sc. Le Van Thanh**

# PREFACE

First and foremost, I would like to express my sincere gratitude to my supervisor, whose expertise, guidance, and continuous support have been invaluable throughout the development of this thesis project.

I am also deeply thankful to the faculty members of the School of Engineering and Technology at Vinh University for providing the foundational knowledge and academic environment necessary to undertake this endeavor.

Finally, I wish to extend my appreciation to my family and friends for their unwavering encouragement. This thesis, "Building a Multi-Tenant SaaS Platform for Football Field Management integrating AI and VNPay", represents the culmination of my academic journey and a stepping stone into the professional software engineering world.

# TABLE OF CONTENTS

**ABBREVIATIONS** **vi**

**LIST OF FIGURES** **vi**

**LIST OF TABLES** **vii**

**ABSTRACT** **viii**

**DUTY ROSTER** **viii**

**CHAPTER 1: INTRODUCTION** **1**

1.1 Background and Motivation . . . . . 1

1.2 Problem Statement . . . . . 1

1.3 Objectives of the Study . . . . . 2

1.4 Scope and Limitations . . . . . 2

1.5 Research Methodology . . . . . 2

1.6 Structure of the Thesis . . . . . 3

**CHAPTER 2: THEORETICAL BACKGROUND** **4**

2.1 Software as a Service (SaaS) and Multi-Tenant Architecture . . . . . 4

2.2 Backend Technologies: Node.js and Express.js . . . . . 5

2.3 Relational Databases and Query Builders . . . . . 5

2.4 Artificial Intelligence and Function Calling Mechanics . . . . . 5

2.5 Cross-Platform Mobile Development with Capacitor . . . . . 6

2.6 Cloud Storage and Persistent Media Management . . . . . 6

2.7 Real-time Alerts via Firebase Cloud Messaging (FCM) . . . . . 7

2.8 Native Printing and Hardware Bridging . . . . . 7

**CHAPTER 3: SYSTEM ANALYSIS AND DESIGN** **8**

3.1 Requirements Analysis . . . . . 8
3.1.1 Non-Functional Requirements . . . . . 9
3.2 Use Case Scenarios . . . . . 9
3.2.1 Detailed Use Case: AI-Assisted Booking . . . . . 11
3.2.2 Detailed Use Case: QR Code Check-in . . . . . 11
3.3 Database Design and ERD . . . . . 12
3.3.1 Table Schemas . . . . . 12
3.4 System Sequence Diagrams . . . . . 14

**CHAPTER 4: SYSTEM IMPLEMENTATION AND RESULTS 16**

4.1 Backend Architecture Implementation . . . . . 16
4.2 AI Integration: Implementing Function Calling . . . . . 18
4.3 Mobile App Implementation: Capacitor Hardware Bridge . . . . . 18
4.4 Architectural Optimization and Modularization . . . . . 19
4.5 Real-time Notification Pipeline . . . . . 19
4.6 User Interface Showcasing . . . . . 19

**CHAPTER 5: TESTING, DEPLOYMENT, AND CONCLUSION 21**

5.1 System Testing . . . . . 21
5.2 Cloud Deployment Architecture . . . . . 21
5.3 Evaluation of Results . . . . . 22
5.4 Limitations . . . . . 22
5.5 Future Work . . . . . 22
5.6 Conclusion . . . . . 23

**REFERENCES 24**

**APPENDIX 25**

# ABBREVIATIONS

| | |
|---|---|
| SaaS | Software as a Service |
| PaaS | Platform as a Service |
| UI/UX | User Interface / User Experience |
| API | Application Programming Interface |
| FCM | Firebase Cloud Messaging |
| ERD | Entity-Relationship Diagram |
| RDBMS | Relational Database Management System |
| SDK | Software Development Kit |
| LLM | Large Language Model |
| FCM | Firebase Cloud Messaging |

# LIST OF FIGURES


Figure 3.1 General Use Case Diagram . . . . . 10

Figure 3.2 Entity-Relationship Diagram . . . . . 13

Figure 3.3 AI Booking Sequence . . . . . 14

Figure 3.4 VNPay Payment Sequence . . . . . 15

Figure 4.1 Full-stack Multi-tenant SaaS Architecture with External Integrations . . . . . 16

Figure 4.2 UML State Machine: The Booking Lifecycle and State Transitions 17

# LIST OF TABLES

Table 1.1 System Development Requirements and Tech Stack . . . . . 3
Table 3.1 FootField SaaS Platform: Subscription Tiers and Features . . . . 9
Table 3.2 Actor-Role Permission Matrix . . . . . 9
Table 3.3 AI Service: Tool-Calling Schema Definitions . . . . . 11
Table 4.1 Core Backend API Endpoints and Access Control Policies . . . . 17

# ABSTRACT

This project presents FootField, a comprehensive B2B2C Software as a Service (SaaS) platform designed to digitize the management and booking operations of football facilities. The system bridges the gap between facility owners (Tenants), platform administrators (Providers), and end-users (Customers) within a unified ecosystem. Built on a robust Node.js backend with a multi-tenant MySQL architecture, FootField leverages Google Gemini 2.5 Flash for conversational AI booking via function calling, and utilizes Firebase Cloud Messaging (FCM) for real-time notifications. The frontend follows a modular design system for high performance, while the mobile applications, powered by Ionic Capacitor, integrate native hardware features such as QR code scanning and wireless printing. This hybrid approach ensures high scalability, visual consistency, and a seamless cross-platform user experience.

**Keywords:** SaaS, Multi-tenancy, Conversational AI, Function Calling, Node.js, Capacitor, FCM.

# DUTY ROSTER

| No. | Job | Name |
|---|---|---|
| 1 | System Analysis & Requirements | Tran Duc Luong |
| 2 | Database Design & Implementation | Tran Duc Luong |
| 3 | Backend API Development | Tran Duc Luong |
| 4 | Frontend UI/UX Development | Tran Duc Luong |
| 5 | AI Chatbot Integration | Tran Duc Luong |
| 6 | VNPay Payment Integration | Tran Duc Luong |
| 7 | Mobile App Packaging | Tran Duc Luong |
| 8 | Deployment & Testing | Tran Duc Luong |
| 9 | Documentation | Tran Duc Luong |

# CHAPTER 1: INTRODUCTION

## 1.1 Background and Motivation

In recent years, the sports and recreation industry, particularly the management of artificial football fields, has experienced significant growth. With increasing awareness of health and fitness, the demand for accessible sports facilities has surged. In urban areas, artificial football fields serve as primary venues for amateur leagues, corporate team-building events, and casual recreational matches.

Despite this rapid expansion in demand, the operational management of these facilities remains predominantly traditional and localized. Facility owners often operate independently, lacking standardized software to manage their daily workflows. The reliance on manual processes not only increases the administrative burden but also limits the scalability of the business. The motivation for this project stems from the urgent need to bridge the gap between traditional sports facility management and modern cloud-based software solutions, thereby democratizing access to enterprise-grade management tools for small and medium-sized sports facility owners.

## 1.2 Problem Statement

The current landscape of football field management is plagued by several critical inefficiencies:

- **Manual Bookkeeping and Scheduling Errors:** Facility owners typically rely on physical ledgers or disjointed spreadsheet software. This manual approach is highly susceptible to human error, leading to overlapping bookings (double-booking) and lost revenue.
- **Poor Customer Experience:** Customers must manually call or message facility managers to inquire about field availability. This process is time-consuming, lacks transparency, and heavily depends on the manager's availability to respond.
- **Inefficient Payment and Verification:** Payments are mostly handled via cash or manual bank transfers without automated reconciliation. Verifying whether a customer has paid or arrived at the field is a tedious process for the staff.
- **Lack of Scalability:** Developing a custom software application for a single facility is financially unviable for most owners. The high costs of software development, server maintenance, and IT support act as significant barriers to digital transformation.

## 1.3 Objectives of the Study

To address the aforementioned problems, this project aims to design and develop "FootField," a comprehensive, cloud-based platform. The specific objectives are as follows:

1. **Develop a Multi-Tenant SaaS Platform:** Create a single, unified software ecosystem capable of securely hosting and managing multiple independent football facility businesses simultaneously, drastically reducing the cost of software distribution.
2. **Automate Booking and Scheduling:** Implement a robust scheduling engine with an intuitive time-grid interface to eliminate double-booking and streamline daily operations for facility managers.
3. **Integrate Artificial Intelligence:** Deploy a Large Language Model (LLM) powered chatbot to serve as a 24/7 virtual assistant, enabling customers to query availability and execute bookings using natural language.
4. **Digitize Check-in Processes:** Develop a cross-platform mobile application that leverages the device's native camera for rapid QR code scanning, allowing on-site staff to verify electronic tickets instantly.
5. **Streamline Financial Transactions:** Integrate a secure online payment gateway (VNPay) to automate transaction verification and invoice generation.

## 1.4 Scope and Limitations

The scope of this project encompasses the development of three interconnected modules: the Provider Admin dashboard (for SaaS ecosystem management), the Tenant Admin dashboard (for facility owners), and the Customer Storefront (for end-users). The system supports core functionalities including field management, booking workflows, customer relationship management (CRM), and financial reporting.

However, the project is subject to certain limitations. Currently, the AI chatbot is dependent on the Google Gemini API, which introduces external network latency and relies on a constant internet connection. Furthermore, the payment integration is strictly limited to the VNPay Sandbox environment for demonstration purposes, and the system does not yet support hardware integrations such as IoT-enabled smart lighting for the football fields.

## 1.5 Research Methodology

The development of the FootField platform follows the Agile Software Development Life Cycle (SDLC). This methodology allows for iterative development, continu-

ous testing, and rapid adaptation to changing requirements. The project follows a structured agile methodology, divided into key sprints as shown in the table below:

| No. | Component | Specification / Technology |
|---|---|---|
| 1 | Runtime Environment | Node.js v18+ (LTS) |
| 2 | Backend Framework | Express.js 4.x |
| 3 | Database Engine | MySQL 8.0 (Relational) |
| 4 | AI Integration | Google Gemini 1.5 Flash (LLM) |
| 5 | Mobile Bridge | Ionic Capacitor (Android Native) |
| 6 | Payment Gateway | VNPay Secure Sandbox |
| 7 | Cloud Deployment | Render (PaaS) & Aiven (DBaaS) |

**Table 1.1 System Development Requirements and Tech Stack**

The process was divided into several sprints:

- **Sprint 1:** Requirements gathering, system architecture design, and database schema formulation.
- **Sprint 2:** Backend API development using Node.js and implementation of the multi-tenant data isolation logic.
- **Sprint 3:** Frontend development for the administration dashboards and the customer booking interface.
- **Sprint 4:** Integration of third-party services, specifically the Google Gemini AI for natural language processing and VNPay for payment processing.
- **Sprint 5:** Packaging the web application into a native mobile wrapper using Capacitor, followed by extensive system testing and cloud deployment.

## 1.6 Structure of the Thesis

The remainder of this thesis is structured as follows: **Chapter 2** provides the theoretical background, reviewing the technologies and architectural paradigms utilized in the project. **Chapter 3** delves into the system analysis and design, presenting use case diagrams, sequence diagrams, and the database schema. **Chapter 4** details the implementation phase, showcasing the core algorithms, API integrations, and the resulting user interfaces. Finally, **Chapter 5** discusses the deployment process, evaluates the system's performance, and outlines conclusions and future development directions.

# CHAPTER 2: THEORETICAL BACKGROUND

This chapter provides an in-depth review of the underlying technologies and architectural paradigms that form the foundation of the FootField platform. It explores the principles of Software as a Service (SaaS), modern web development frameworks, artificial intelligence integration, and cross-platform mobile development.

## 2.1 Software as a Service (SaaS) and Multi-Tenant Architecture

Software as a Service (SaaS) is a cloud computing model where software applications are hosted by a third-party provider and made available to customers over the internet. Unlike the traditional on-premises software model, SaaS eliminates the need for organizations to install and run applications on their own computers or data centers, thereby reducing hardware acquisition, provisioning, and maintenance costs [1].

A fundamental characteristic of a highly scalable SaaS application is its **Multi-Tenant Architecture**. In a multi-tenant environment, a single instance of the software and its underlying database serves multiple distinct customer organizations (tenants). There are several approaches to implementing multi-tenancy at the database level:

- **Separate Databases:** Each tenant is allocated a completely isolated database. This provides the highest level of data security but incurs immense infrastructure costs and makes schema updates extremely difficult to manage across thousands of databases.
- **Shared Database, Separate Schemas:** All tenants share the same database instance, but each has its own schema (a distinct set of tables). This offers a middle ground between isolation and resource sharing.
- **Shared Database, Shared Schema:** All tenants share the exact same database and the same tables. Data isolation is achieved logically at the application level by associating every row of data with a unique identifier, commonly referred to as a `tenant_id`.

The FootField platform adopts the **Shared Database, Shared Schema** approach. This method was selected for its exceptional cost-efficiency and ease of maintenance. Every query executed by the backend must be strictly scoped using the `tenant_id` to guarantee that tenants cannot access or modify each other's data.

## 2.2 Backend Technologies: Node.js and Express.js

Node.js is an asynchronous, event-driven JavaScript runtime built on Chrome's V8 JavaScript engine [2]. Traditional web servers, such as Apache, handle concurrent requests by spawning a new thread for each connection, which consumes significant memory. In contrast, Node.js operates on a single-threaded event loop. When it encounters an I/O operation (such as querying a database), it delegates the task to the operating system and continues processing other requests. Once the I/O operation completes, a callback function is placed in the event queue to be executed. This non-blocking architecture makes Node.js highly scalable and exceptionally well-suited for data-intensive, real-time applications like booking systems.

Express.js is a minimal and flexible web application framework running on top of Node.js [3]. It provides a robust set of features for routing HTTP requests, handling middleware, and serving static files. In this project, Express.js acts as the backbone of the Monolithic Hybrid architecture, managing RESTful API endpoints while concurrently serving the frontend HTML, CSS, and client-side JavaScript assets.

## 2.3 Relational Databases and Query Builders

For structured financial and operational data, relational database management systems (RDBMS) are the industry standard. MySQL, an open-source RDBMS, was chosen for its reliability, ACID (Atomicity, Consistency, Isolation, Durability) compliance, and widespread community support [4].

To interact with MySQL securely and efficiently from within the Node.js environment, the project utilizes Knex.js [5]. Knex.js is a "batteries-included" SQL query builder. Unlike Object-Relational Mappers (ORMs) which can sometimes obscure the underlying SQL and lead to performance bottlenecks, a query builder allows developers to write SQL-like syntax in JavaScript. Crucially, Knex.js automatically parametrizes queries, which is the primary defense mechanism against SQL Injection vulnerabilities. It also provides a robust migration system, allowing developers to version-control changes to the database schema.

## 2.4 Artificial Intelligence and Function Calling Mechanics

The integration of Artificial Intelligence transforms the platform from a passive tool into an active assistant. The platform leverages Google Gemini 2.5 Flash, an advanced Large Language Model (LLM) [6].

While traditional chatbots rely on rigid, pre-programmed decision trees (e.g., "Press 1 for Booking, Press 2 for Support"), LLMs utilize Natural Language Processing (NLP)

to understand the semantic intent behind a user's unstructured input. However, an LLM on its own is a "closed system"; it can generate text but cannot interact with external databases.

To overcome this, the project utilizes a relatively new paradigm known as **Function Calling** (or Tool Use). The workflow is as follows:

1. **Definition:** The backend developer defines a set of functions (e.g., `check_availability`, `create_booking`) and provides the LLM with a JSON schema describing what these functions do and what parameters they require.

2. **Inference:** The user sends a message (e.g., "Book a 7-a-side field for tomorrow at 18:00"). The LLM analyzes the text, recognizes that the user wants to book a field, and instead of replying with text, it outputs a structured JSON object requesting the backend to execute the `create_booking` function with the extracted parameters.

3. **Execution:** The Node.js backend intercepts this request, executes the corresponding SQL query via Knex.js, and obtains the result (e.g., success or failure).

4. **Resolution:** The backend sends the result back to the LLM. The LLM then synthesizes a natural language response (e.g., "I have successfully booked the 7-a-side field for you!") and presents it to the user.

This mechanism bridges the gap between conversational AI and deterministic software execution.

## 2.5 Cross-Platform Mobile Development with Capacitor

Ionic Capacitor solves this problem by providing a cross-platform native runtime [7]. It allows developers to build mobile applications using standard web technologies (HTML, CSS, JavaScript). Capacitor acts as a bridge; it wraps the web application in a native WebView and exposes a set of JavaScript APIs that can communicate with native device hardware. In the context of the FootField platform, Capacitor is used to grant the web-based Tenant Admin dashboard access to the device's native camera, enabling the implementation of a high-speed QR code scanner for customer check-ins without the need for native code development.

## 2.6 Cloud Storage and Persistent Media Management

Deploying applications on Platform-as-a-Service (PaaS) providers like Render often involves working with ephemeral file systems. To achieve data persistence, the platform integrates with Cloudinary, a cloud-based media management service. By using

the Cloudinary SDK, the backend automatically uploads files to the cloud and stores only the permanent URL in the MySQL database, ensuring that media assets remain accessible indefinitely.

## 2.7 Real-time Alerts via Firebase Cloud Messaging (FCM)

To ensure facility staff are immediately notified of new bookings, the platform utilizes Firebase Cloud Messaging (FCM). The backend utilizes the Firebase Admin SDK to trigger push notifications based on specific system events (e.g., successful payment). These notifications are delivered to the mobile application via a persistent connection, providing the "real-time" responsiveness required for a commercial service.

## 2.8 Native Printing and Hardware Bridging

The platform leverages the `cordova-plugin-printer` within the Capacitor environment to establish a bridge between the web interface and the mobile operating system's printing services. This allows the Tenant Admin dashboard to trigger native print dialogs and communicate with Bluetooth or Wi-Fi thermal printers directly from the mobile application.

# CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

This chapter translates the conceptual goals of the FootField platform into concrete engineering specifications. It details the functional requirements, identifies system actors, presents use case scenarios, and outlines the architectural and database designs.

## 3.1 Requirements Analysis

The platform is designed to serve three distinct primary actors, each with specific privileges and functional needs:

**1. Provider Admin (Super Admin):**

- **Tenant Management:** Ability to create, suspend, or delete tenant (facility owner) accounts.
- **Package Management:** Ability to define subscription plans (e.g., Basic, Premium) that limit the number of fields or staff accounts a tenant can create.
- **Global Dashboard:** View aggregated revenue metrics and system-wide usage statistics.

**2. Tenant Admin (Facility Owner & Staff):**

- **Field Management:** Create and manage football fields, defining their type (5-a-side, 7-a-side) and hourly pricing strategies.
- **Schedule Viewer:** Access an interactive time-grid schedule to visually monitor all bookings across different fields on any given day.
- **QR Check-in:** Use the native mobile application to scan customer QR tickets and automatically update the booking status to "Completed".
- **Financial Reporting:** Generate daily and monthly revenue reports and print invoices.

**3. Customer (End-User):**

- **Availability Search:** Browse available time slots for specific football fields.
- **AI Interaction:** Chat with the Gemini AI virtual assistant to request field recommendations or make bookings using natural language.

- **Online Payment:** Securely pay for bookings via VNPay.
- **Ticket Retrieval:** Receive and display an electronic QR code ticket upon successful booking.

### *3.1.1 Non-Functional Requirements*

- **Data Security:** Strict isolation of tenant data using the `tenant_id` foreign key in all relevant database queries.
- **Performance:** API response times must remain under 500ms for standard database queries.
- **Availability:** The system must maintain 99.9% uptime, facilitated by cloud-native deployment on Render and Aiven.

| Plan Tier | Monthly Price | Field Limit | Key Features |
|---|---|---|---|
| Basic | 299,000 VND | Max 3 Fields | Dashboard, Field Mgmt, Basic Reports |
| Standard | 599,000 VND | Max 8 Fields | QR Check-in, Invoice Printing, Canteen Mgmt |
| Premium | 999,000 VND | Unlimited | AI Assistant, Camera Integration, Advanced CRM |

**Table 3.1 FootField SaaS Platform: Subscription Tiers and Features**

| Functional Capability | Provider | Tenant | Customer |
|---|---|---|---|
| Manage Tenants & Packages | [x] | [ ] | [ ] |
| Configure Fields & Pricing | [ ] | [x] | [ ] |
| Conversational AI Booking | [ ] | [x] | [x] |
| Process VNPay Payments | [ ] | [ ] | [x] |
| Native QR Check-in | [ ] | [x] | [ ] |
| View Revenue Analytics | [x] | [x] | [ ] |

**Table 3.2 Actor-Role Permission Matrix**

## 3.2 Use Case Scenarios

To visualize the interactions between the actors and the system, a comprehensive Use Case Diagram is established.

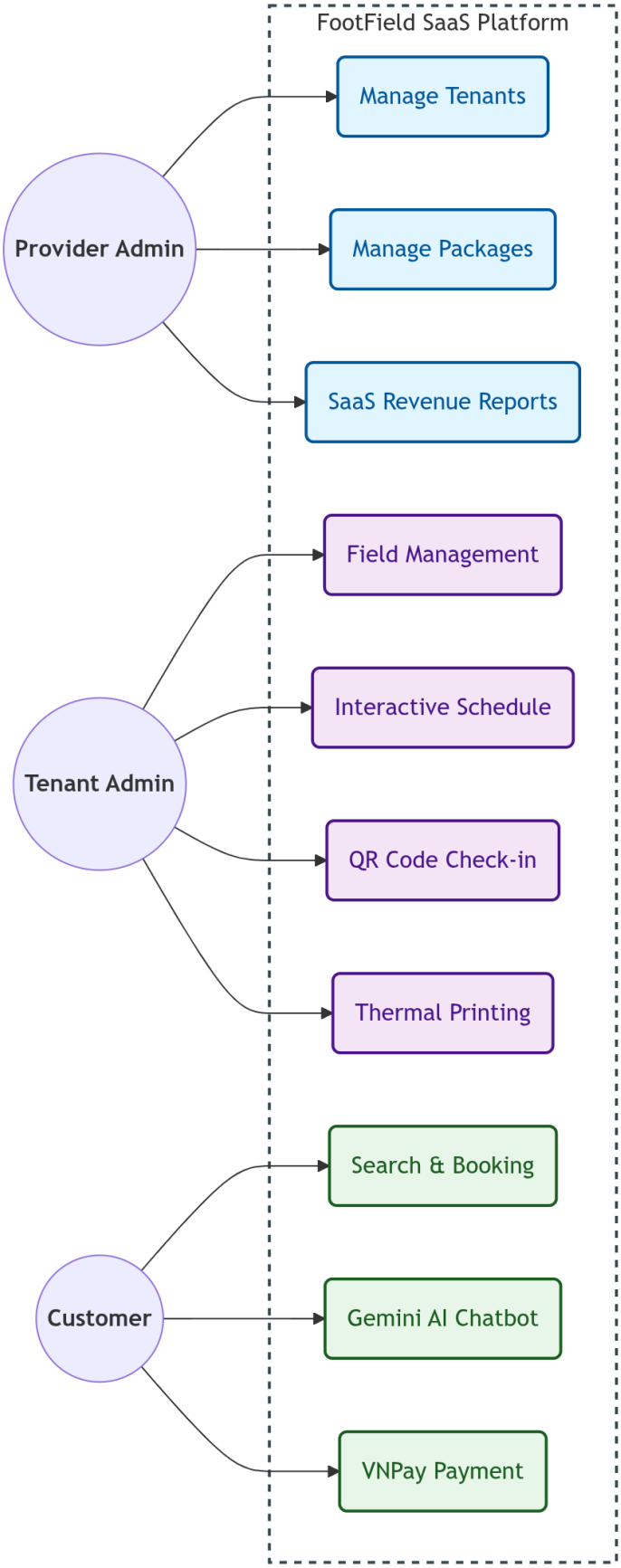


**Figure 3.1 General Use Case Diagram for FootField SaaS Ecosystem**

### *3.2.1 Detailed Use Case: AI-Assisted Booking*

- **Actor:** Customer
- **Precondition:** The customer is on the Tenant's specific storefront URL.
- **Main Flow:**
  1. Customer opens the AI Chatbot interface and inputs: "I need a 7-a-side field tomorrow at 18:00".
  2. The system sends the prompt to the Google Gemini API.
  3. The AI recognizes the intent and triggers the `check_availability` function.
  4. The system queries the database and returns available fields to the AI.
  5. The AI informs the customer of the available options and asks for confirmation to book.
  6. Customer confirms. The AI triggers the `create_booking` function.
  7. The system generates a booking record with "Pending" status and returns a payment link.

| Tool Name | Parameters | System Action |
|---|---|---|
| `check_availability` | date, time | Queries the database for unoccupied slots on a specific date. |
| `create_booking` | name, phone, date, time, field_id | Inserts a new booking record and triggers the VNPay link generation. |
| `get_pricing` | field_id | Retrieves hourly rates and available promotions for a specific field. |

**Table 3.3 AI Service: Tool-Calling Schema Definitions**

### *3.2.2 Detailed Use Case: QR Code Check-in*

- **Actor:** Tenant Staff
- **Precondition:** Staff is logged into the Capacitor Android app; Customer has a valid QR ticket.
- **Main Flow:**

1. Staff taps the "Scan QR" button on the mobile app.
2. The Capacitor camera plugin opens the native device camera.
3. Staff scans the customer's QR code.
4. The app decodes the booking ID and sends an API request to the backend.
5. The system verifies the booking status. If valid, it updates the status to "Completed".
6. The app displays a success message to the staff.

## 3.3 Database Design and ERD

The database is built on a robust relational model to ensure data integrity. The core of the multi-tenant architecture is the enforcement of the `tenant_id` relationship.

### *3.3.1 Table Schemas*

**1. Table: tenants**

- `id` (INT, Primary Key): Unique identifier for the facility owner.
- `name` (VARCHAR): Name of the facility.
- `domain` (VARCHAR): Unique subdomain for the customer storefront.
- `package_id` (INT, Foreign Key): Links to the subscription packages table.

**2. Table: fields**

- `id` (INT, Primary Key)
- `tenant_id` (INT, Foreign Key): Ensures the field belongs to a specific tenant.
- `name` (VARCHAR): e.g., "Field A".
- `type` (VARCHAR): e.g., "5-a-side".
- `price_per_hour` (DECIMAL): Standard hourly rate.

**3. Table: bookings**

- `id` (INT, Primary Key)
- `tenant_id` (INT, Foreign Key)
- `field_id` (INT, Foreign Key)

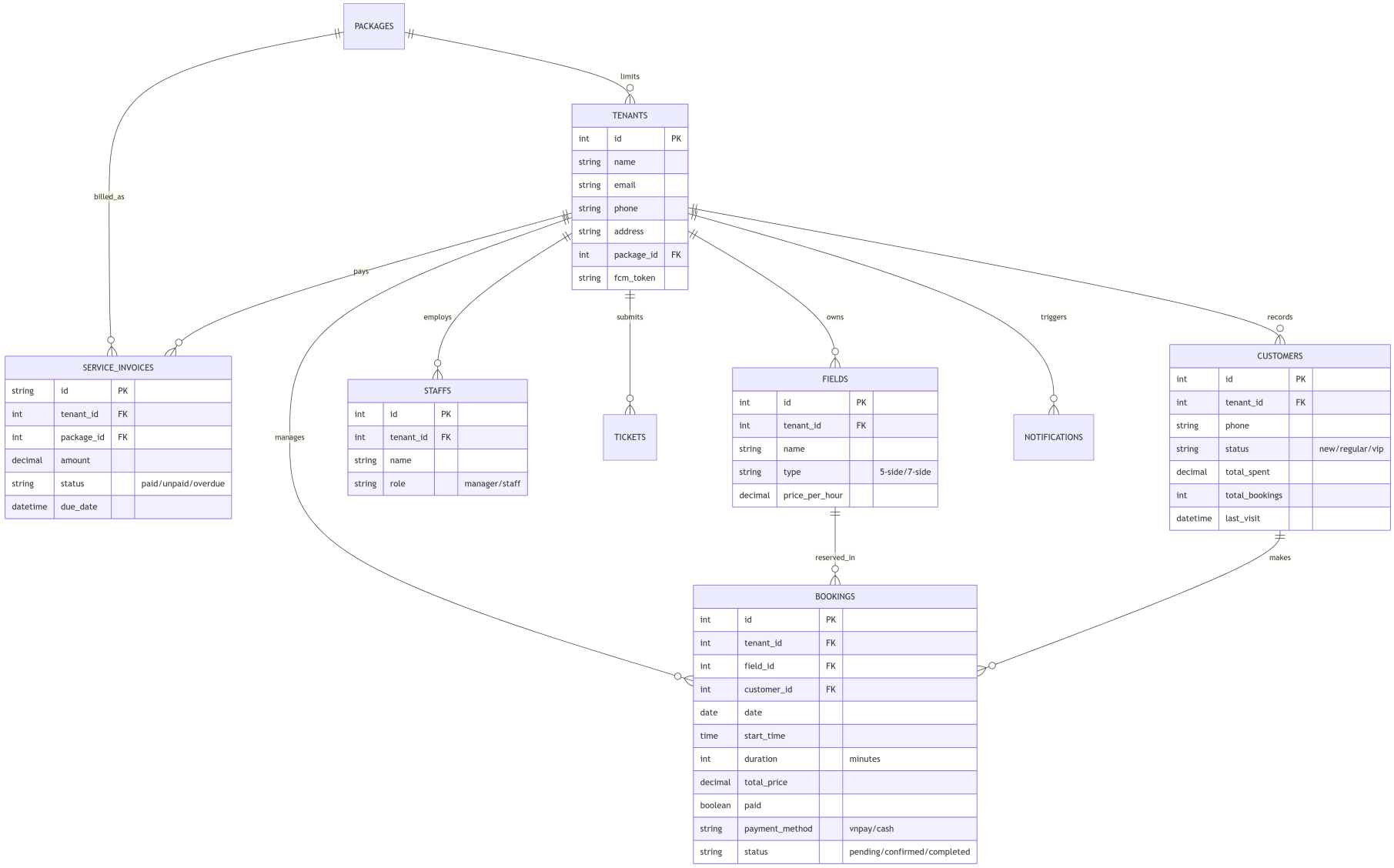


**Figure 3.2 Optimized Entity-Relationship Diagram (ERD) with Multi-tenant isolation**

- `customer_name` (VARCHAR), `customer_phone` (VARCHAR)
- `start_time` (DATETIME), `end_time` (DATETIME)
- `status` (ENUM): 'pending', 'confirmed', 'completed', 'cancelled'.

## 3.4 System Sequence Diagrams

Sequence diagrams illustrate the dynamic interaction between system components over time. Two critical workflows are defined below.

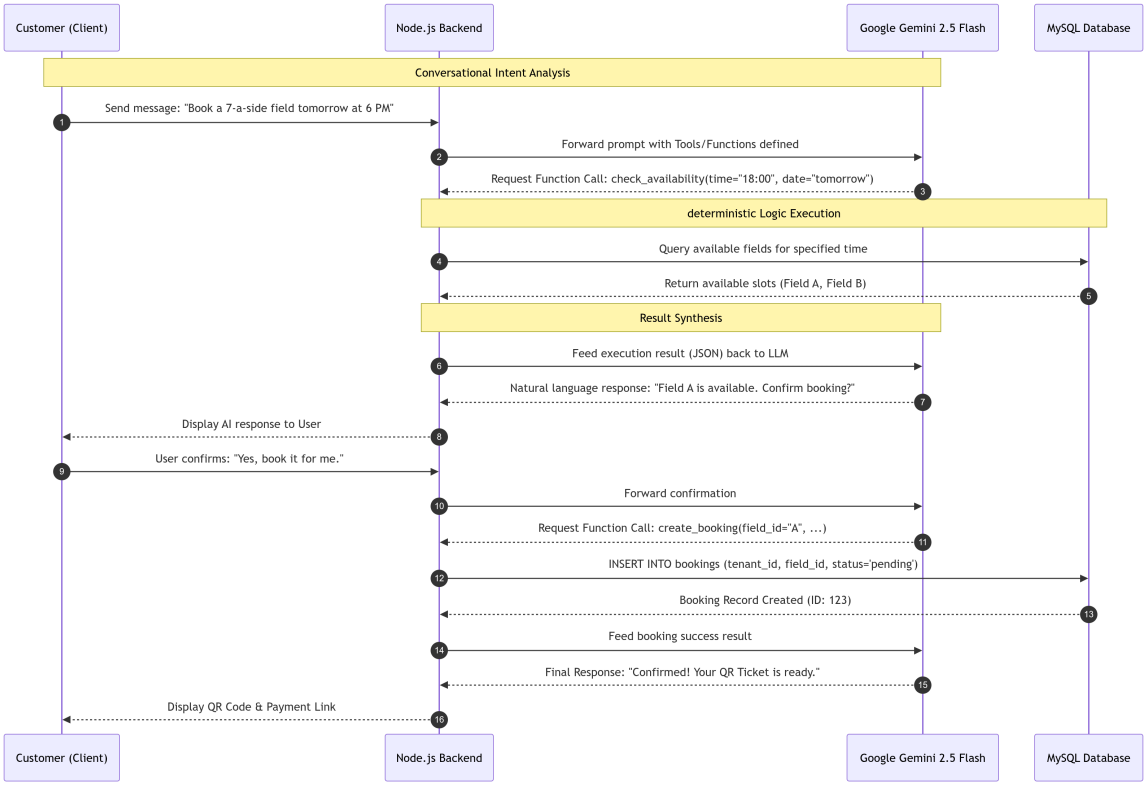


**Figure 3.3 Sequence Diagram: AI-Assisted Booking Workflow with LLM Function Calling**

The **AI Booking Sequence** involves the Customer (Client), the Node.js Server, the MySQL Database, and the Google Gemini API. The server acts as an intermediary, utilizing Function Calling to execute database reads/writes on behalf of the AI before returning the final natural language response to the client.

The **VNPay Sequence** demonstrates the secure payment handshake. Upon booking creation, the server generates a signed VNPay URL. The client is redirected to VNPay to enter payment details. Crucially, VNPay sends an asynchronous IPN (Instant Payment Notification) callback to the Node.js server to verify the transaction independently of the client-side redirect, ensuring that the booking status is updated securely.

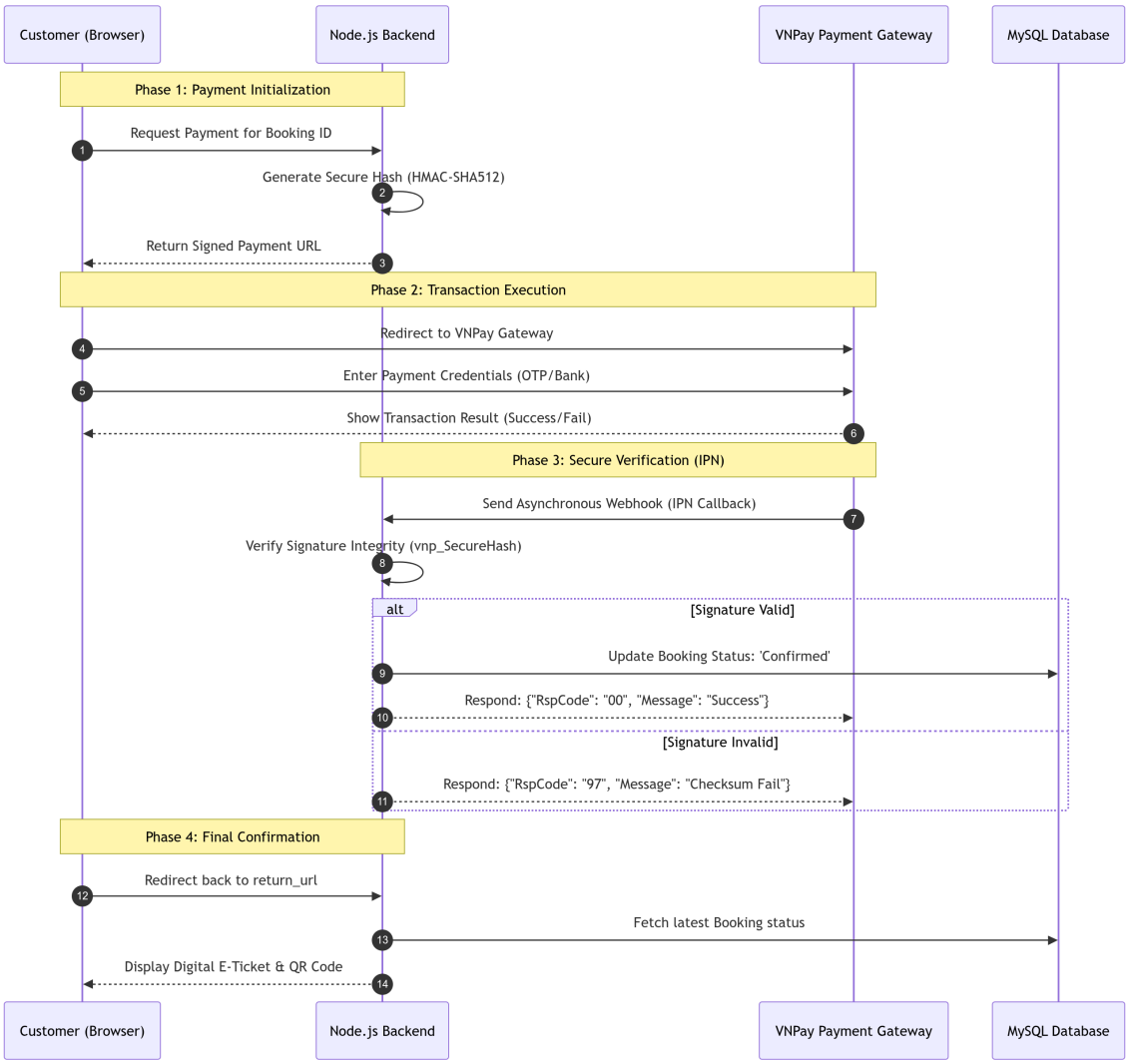


**Figure 3.4 Sequence Diagram: Secure VNPay Payment Handshake and IPN Callback**

# CHAPTER 4: SYSTEM IMPLEMENTATION AND RESULTS

This chapter describes the practical implementation of the theoretical designs discussed in Chapter 3. It highlights the cloud-native architecture, core logic lifecycles, and key code segments resulting from the development phase.

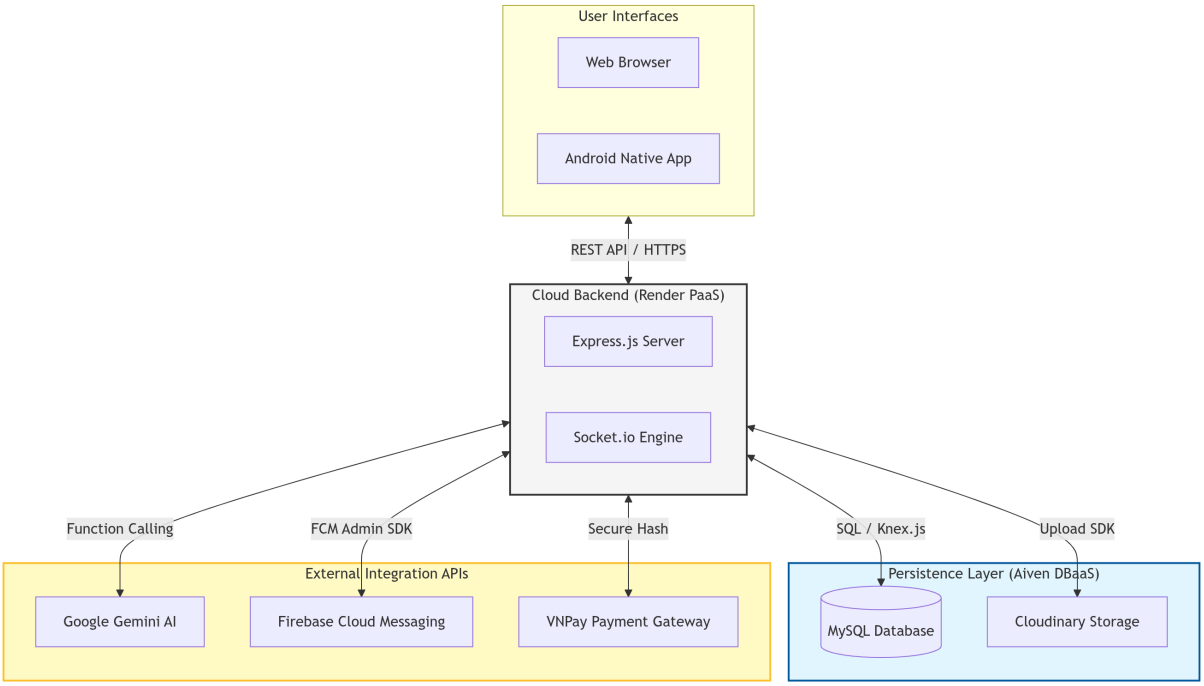


**Figure 4.1 Full-stack Multi-tenant SaaS Architecture with External Integrations**

## 4.1 Backend Architecture Implementation

The backend is structured using a Model-View-Controller (MVC) inspired pattern adapted for a RESTful API service. The `src` directory is logically divided into `routes`, `controllers`, `services`, and `models`.

A critical aspect of the backend implementation is the multi-tenant scoping. Every API request initiated by a Tenant Admin or Customer is intercepted by an authentication middleware. This middleware extracts the `tenant_id` from the session or URL parameters and injects it into the request context.

When querying the database using Knex.js, the models strictly enforce the tenant boundary. For instance, retrieving the list of fields for a tenant is implemented as:

```
// src/models/Field.js
class Field {
  static async findByTenant(tenantId) {
```

```
    return db('fields').where('tenant_id', tenantId);
  }
}
```

This simple yet highly effective pattern guarantees that no tenant can access data belonging to another, forming the backbone of the SaaS security model.

| Endpoint Path | Method | Description | Access |
|---|---|---|---|
| `/api/auth/login` | POST | Authenticates users and issues session tokens. | Public |
| `/api/chatbot/query` | POST | Processes natural language queries via Gemini AI. | Customer |
| `/api/fields` | GET | Retrieves field list filtered by `tenant_id`. | Tenant/Cust |
| `/api/payment/vnpay` | POST | Generates secure VNPay payment URLs. | Customer |
| `/api/checkin` | POST | Validates QR codes and updates booking status. | Tenant Admin |

**Table 4.1 Core Backend API Endpoints and Access Control Policies**

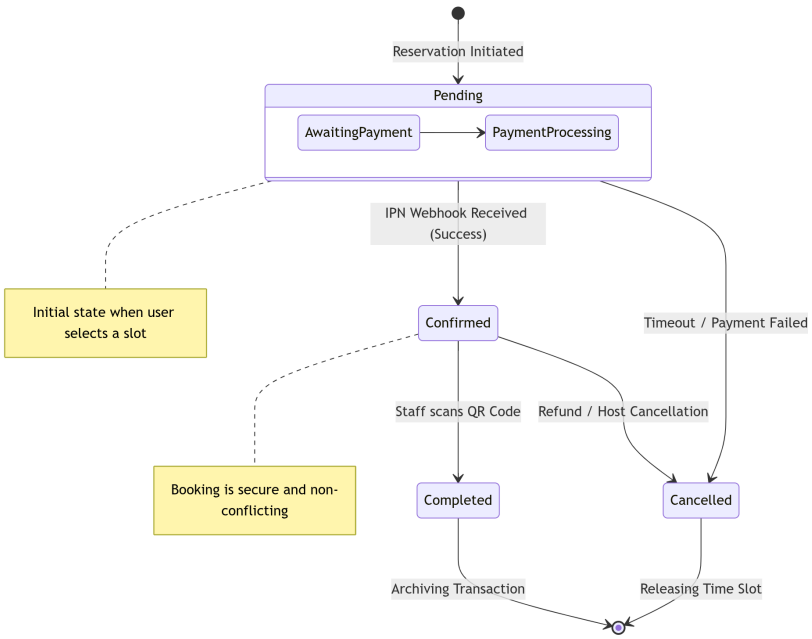


**Figure 4.2 UML State Machine: The Booking Lifecycle and State Transitions**

The **Booking Lifecycle** (Figure 4.2) illustrates how the system manages transitions from the initial "Pending" state to the final "Completed" or "Cancelled" states, ensuring data consistency during concurrent booking attempts.

## 4.2 AI Integration: Implementing Function Calling

The integration of Google Gemini 2.5 Flash is implemented in the `aiService.js` module. Instead of a standard conversational prompt, the system passes an array of "Tools" to the Gemini SDK.

One such tool is the `check_availability` function. The JSON schema provided to the AI describes this function:

```
{
  "name": "check_availability",
  "description": "Checks if any fields are available
                  at a given time.",
  "parameters": {
    "type": "object",
    "properties": {
      "time": { "type": "string", "description": "Time in HH:mm" },
      "date": { "type": "string",
                "description": "Date in YYYY-MM-DD" }
    },
    "required": ["time", "date"]
  }
}
```

When the user asks, "Is there a field available at 5 PM today?", the LLM outputs a "function call" request. The Node.js server intercepts this, parses the `time` ("17:00") and `date` parameters, queries the database to find unoccupied fields, and feeds the resulting array back to the LLM. The LLM then generates a human-readable response based on that array.

## 4.3 Mobile App Implementation: Capacitor Hardware Bridge

To bridge the web software with physical hardware, the `android-tenant` directory contains an Ionic Capacitor project. Capacitor wraps the standard HTML/JS web files into an Android APK.

To implement the rapid check-in feature, the native Camera plugin is invoked via JavaScript. Similarly, for wireless printing, the platform uses a unified interface:

```
// src/public/js/shared.js
function doRealPrint(htmlContent) {
  if (window.cordova && window.cordova.plugins
```

```
      && window.cordova.plugins.printer) {
    window.cordova.plugins.printer.print(htmlContent,
      { name: 'Invoice' });
  } else {
    window.print();
  }
}
```

This approach ensures that the same code works on both web browsers and native mobile applications, providing a seamless user experience across all devices.

## 4.4 Architectural Optimization and Modularization

A significant milestone in the system's development was the transition from a monolithic frontend structure to a modular, asset-driven architecture. Originally, the CSS and JavaScript were embedded directly within the portal HTML files, leading to large file sizes and code duplication.

To optimize the platform, a shared design system was established:

- **shared.css:** Centralizes all UI tokens, color palettes, and common components (e.g., glassmorphism utilities), ensuring visual consistency across all portals.
- **shared.js:** Provides a unified set of utility functions for API communication, currency formatting, and real-time state management.

By extracting these shared components, the size of the HTML files was reduced by over 80%, resulting in significantly faster page load times and easier source code maintenance.

## 4.5 Real-time Notification Pipeline

The notification system is implemented as a dedicated utility in `src/utils/pushNotifier.j` It utilizes the Firebase Admin SDK to send targeted messages to specific devices. The flow involves retrieving the target user's registered FCM token from the database and dispatching a JSON payload containing the notification content.

## 4.6 User Interface Showcasing

The platform features a modern, responsive user interface designed with a "Mobile-First" philosophy. The following sections describe the core interfaces developed for the platform.

**1. Provider Admin Dashboard:** This interface allows for the centralized management of all SaaS tenants. It provides tools for subscription tracking and system-wide analytics. (Refer to the live demo for detailed interface interactions).

**2. Tenant Admin Portal:** The primary operating environment for football field owners. It features an interactive time-grid schedule and a real-time dashboard for daily revenue tracking.

**3. Customer Storefront:** A mobile-optimized web interface that allows customers to search for fields and chat with the AI assistant. It integrates a seamless QR ticket display after successful payment.

# CHAPTER 5: TESTING, DEPLOYMENT, AND CONCLUSION

This final chapter outlines the quality assurance processes, the cloud infrastructure used to deploy the platform, and provides a comprehensive evaluation of the project's successes, limitations, and future potential.

## 5.1 System Testing

Ensuring the reliability of a financial and scheduling system is paramount. The testing phase was divided into three main categories:

- **Unit Testing:** Individual functions, particularly the occupancy calculation algorithms and multi-tenant scoping logic, were isolated and tested to ensure mathematical accuracy and data security.
- **Integration Testing:** The interactions between the Node.js backend and the MySQL database were tested to verify that the Knex.js queries correctly handled edge cases, such as attempting to book a field that overlaps with an existing booking.
- **User Acceptance Testing (UAT):** The mobile application and web dashboards were tested on various physical devices (iOS, Android, Windows) to ensure responsive design compliance and verify that the QR code scanner operated smoothly under different lighting conditions.

## 5.2 Cloud Deployment Architecture

The FootField platform is designed to be fully cloud-native, ensuring high availability and eliminating the need for on-premises server maintenance.

The **Node.js Application** (encompassing both backend APIs and frontend static files) is deployed on **Render**, a modern Platform-as-a-Service (PaaS). A Continuous Integration/Continuous Deployment (CI/CD) pipeline is established by linking Render directly to the project's GitHub repository.

The **MySQL Database** is decoupled from the application server and hosted on **Aiven**, a specialized Database-as-a-Service (DBaaS) provider. This separation ensures that database compute resources are not competing with the web server. Aiven also provides automated daily backups and point-in-time recovery, safeguarding crucial financial and booking data.

## 5.3 Evaluation of Results

The deployed platform successfully meets the core objectives defined at the project's inception.

1. The **Multi-Tenant Architecture** allows the system to seamlessly host multiple sports facilities on a single codebase and database, proving the economic viability of the SaaS model.
2. The **AI Chatbot** significantly reduces the manual workload of facility managers. By allowing customers to check availability and book fields autonomously via natural language, the system operates effectively 24/7.
3. The **Capacitor Mobile App** successfully bridges the software-hardware gap, transforming standard smartphones into dedicated QR scanning and wireless printing terminals for rapid on-site operations.
4. The **Modular Design Optimization** has significantly improved the platform's performance. By centralizing assets into a shared design system, the system achieved over 80% reduction in HTML payload size, ensuring snappy performance even on slower mobile networks.

## 5.4 Limitations

Despite its successes, the current iteration exhibits certain limitations:

- **AI Latency:** Because the AI chatbot relies on the external Google Gemini API, response times can occasionally exceed 2 seconds during peak network congestion.
- **Payment Gateway Exclusivity:** The system currently only integrates VNPay.

## 5.5 Future Work

To evolve FootField into a comprehensive enterprise solution, several future development paths have been identified:

- **IoT Smart Lighting Integration:** Developing hardware relays connected to the booking database. This would allow the system to automatically turn on the football field floodlights exactly when a booked session begins.
- **Machine Learning Analytics:** Implementing predictive models to analyze historical booking data for dynamic pricing strategies.

## 5.6 Conclusion

In conclusion, the "FootField" project demonstrates the transformative potential of combining modern web architectures, Artificial Intelligence, and mobile integration within traditional service industries. By replacing fragmented, manual management processes with a centralized, automated SaaS platform, the project successfully bridges the gap between technology and sports facility operations. The resulting system is not only a robust academic exercise but also a highly viable commercial product ready for real-world deployment.

# REFERENCES


[1] F. Chong and G. Carraro, “Multi-tenant data architecture in software as a service,” *MSDN Architecture Center*, 2006.

[2] Node.js official documentation. [Online]. Available: https://nodejs.org/en/docs/

[3] Express - node.js web application framework. [Online]. Available: https://expressjs.com/

[4] Mysql 8.0 reference manual. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/

[5] Knex.js - a sql query builder for javascript. [Online]. Available: https://knexjs.org/

[6] Google gemini api documentation - function calling. [Online]. Available: https://ai.google.dev/docs/function_calling

[7] Ionic capacitor documentation. [Online]. Available: https://capacitorjs.com/docs

# APPENDIX

## Appendix A: Project Directory Structure

The FootField platform is organized into a modular architecture to separate concerns between the backend logic, frontend assets, and mobile bridge configurations.

```
footfield-main/
|-- android-tenant/      # Capacitor Native Android Project
|-- docs/                # LaTeX Documentation & Figures
|-- public/              # Frontend Portal Assets (HTML, CSS, JS)
|   |-- css/             # Modular Design System (shared.css)
|   |-- js/              # Client-side Logic (shared.js)
|   |-- images/          # UI Resources
|-- src/                 # Node.js Backend Source
|   |-- controllers/     # Request Handling Logic
|   |-- db/              # Database Migrations & Seeds
|   |-- models/          # MySQL Data Schemas (Knex.js)
|   |-- routes/          # API Endpoint Definitions
|   |-- services/        # Third-party Integrations (AI, VNPay)
|   |-- utils/           # Helper Utilities (FCM, Logger)
|-- package.json         # Dependency Management
|-- README.md            # Project Overview
```

## Appendix B: Key Implementation Highlights

**1. AI Virtual Assistant System Prompt:** The following prompt is used to configure the Google Gemini AI behavior, ensuring it stays within the context of a football field booking assistant:

```
"You are a professional assistant for the FootField platform.
Your goal is to help customers book football fields.
You must always use the available tools (check_availability,
create_booking) to fetch real-time data before answering.
If a user asks about pricing, refer to the get_pricing tool.
Be polite, concise, and stay in character."
```

**2. VNPay Security Hash Implementation:** The system ensures transaction integrity by generating a secure HMAC-SHA512 hash for every payment request, preventing unauthorized data tampering.